OPERATING SYSTEMS

Project

SUBMITTED BY:

M. Arham BSCE22007

Ahmad Waleed Akhtar BSCE22003

Contents

Introduction

This project demonstrates the implementation of a multithreaded web crawler in C, designed to create a histogram of words starting with each letter of the English alphabet. The program utilizes file handling, multithreaded programming, and performance optimization. Using libcurl for HTTP requests and pthreads for thread management, the crawler processes URLs from a text file and analyzes the data efficiently.

Methodology

The program is executed via a command-line interface with three arguments:
1. The path to a text file containing URLs (one per line).
2. The number of threads for parallel processing.
3. The block size (number of words to process before updating shared data).

The methodology includes the following steps:
- Reading URLs from the input file.
- Dividing URLs among threads for parallel processing.
- Fetching content using libcurl and processing it in blocks.
- Synchronizing shared data updates using pthread mutexes to ensure thread safety.
- Timing various components to evaluate performance.

Results and Discussion

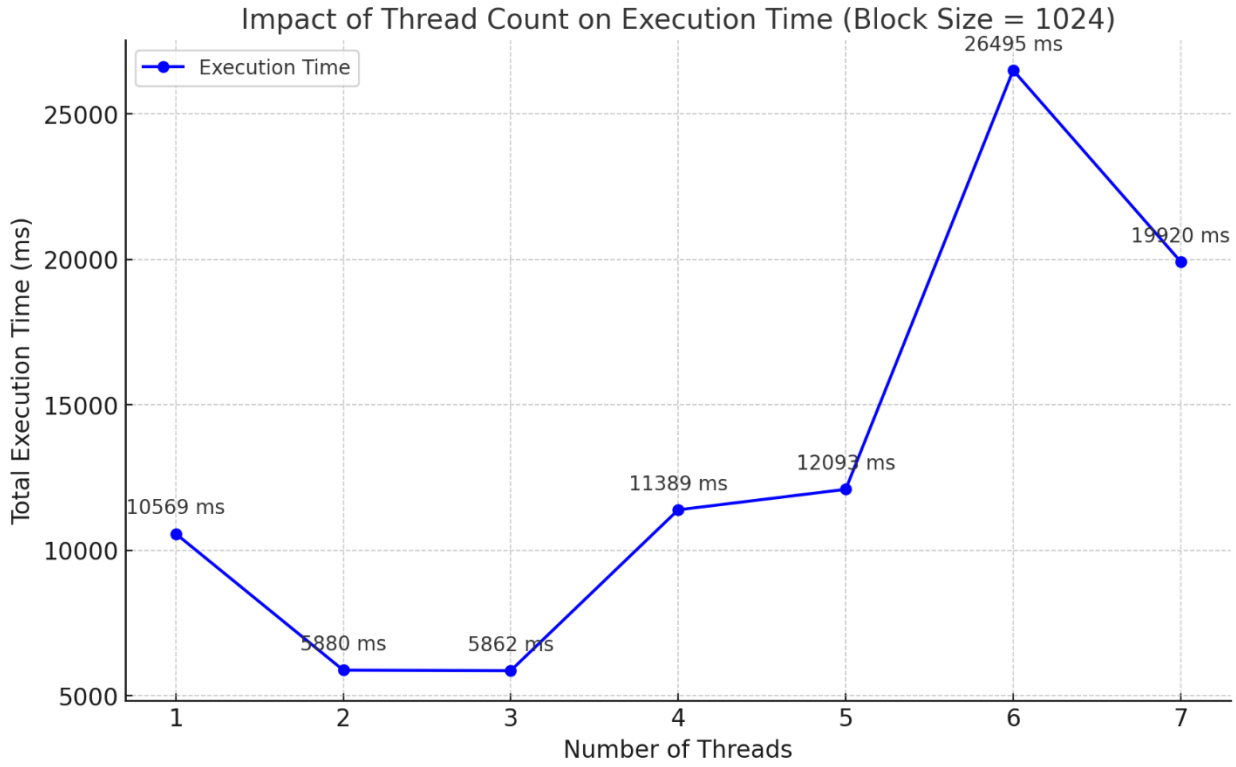
The performance of the web crawler was evaluated using different configurations of threads and block sizes. Timing data was collected for file reading, thread creation, thread joining, and total execution time. The results are presented in the table below.

Timing Results

Number of Threads	Block Size	File Read Time (ms)	Thread Creation Time (ms)	Thread Join Time (ms)	Total Execution Time (ms)
1	1	27	0	42446	42474
1	32	4	0	46077	46081
1	64	5	0	34941	34947
1	256	9	0	44658	44668
1	1024	5	0	35442	35447
3	1	20	1	8667	8688
3	32	4	0	10123	10127
3	64	5	0	3851	3856
3	256	14	1	4216	4231
3	1024	6	1	5135	5142
5	1	16	1	20138	20155
5	32	31	18	9162	9211
5	64	7	1	8951	8959
5	256	17	4	16451	16473
5	1024	12	1	7851	7865

7	1	23	1	7471	7495
7	32	32	1	10387	10420
7	64	14	6	7431	7434
7	256	11	0	7047	7058
7	1024	12	1	13378	13391

A graph comparing the number of threads and total execution time was plotted for block size = 1024. The graph illustrates the impact of increasing thread count on performance.



Factors Affecting Timing Differences

1. **Number of Threads**:
    a. Increasing threads can improve parallelism, reducing **thread join time** as tasks are divided across threads.
    b. However, as threads increase, **overheads** (thread creation and synchronization) become more prominent.
2. **Block Size**:
    a. Larger block sizes reduce the overhead of frequent file reads, as more data is processed per operation.
    b. However, large block sizes may lead to inefficient use of resources when the workload is unbalanced.

Detailed Observations

- **1 Thread:**
  - For block size `1`, the program has the highest **thread join time (42446 ms)** due to lack of parallelism. The thread processes one unit at a time, creating significant overhead.
  - Increasing the block size reduces the **join time**, as fewer I/O operations are required to process the same data. For block size `1024`, the total execution time drops to **35447 ms**.
- **3 Threads:**
  - Adding threads improves parallelism. For block size 1, the execution time drops significantly to **8688 ms** (compared to 42474 ms for 1 thread).
  - At block size 64, the join time is lowest (**3851 ms**) because the workload is evenly distributed, and there is minimal synchronization overhead.
  - Larger block sizes (e.g., 1024) slightly increase execution time (**5142 ms**) due to reduced benefits of parallelism (fewer divisions of work).
- **5 Threads:**
  - Performance is optimal at moderate block sizes, such as 64 (8959 ms) and 1024 (7865 ms), where work is distributed effectively among threads.
  - Larger block sizes (256 or 1024) exhibit diminishing returns due to increased synchronization costs.
  - Smaller block sizes, such as 1, result in higher execution times (20155 ms) due to thread contention.

Code Optimization

1. **Thread Synchronization:** Minimized mutex lock duration by aggregating data locally before updating the shared global structure.
2. **Batch Processing:** Used configurable block sizes to reduce synchronization overhead and balance memory usage.
3. **Dynamic Thread Management:** Adjusted thread counts to align with workload and hardware capabilities for optimal parallelism.
4. **Efficient HTTP Requests:** Leveraged libcurl for lightweight and robust HTTP handling with error management.
5. **Memory Management:** Dynamically allocated and cleaned up memory to handle varying workloads and prevent leaks.

Conclusion

This project successfully implemented a multithreaded web crawler capable of analyzing text data from multiple URLs. The timing results indicate a trade-off between thread count and execution time, where optimal performance is achieved by balancing thread overhead with parallel efficiency.